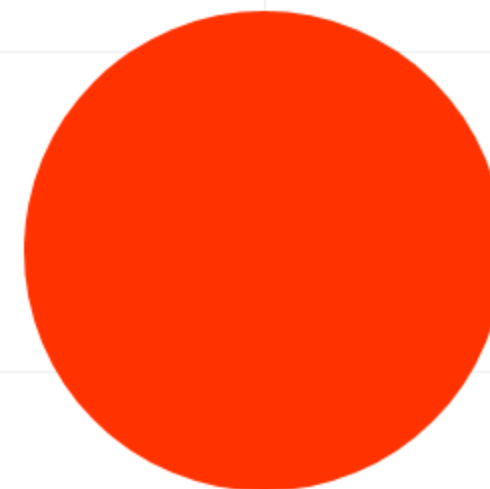


05

MYSQL ADVANCED

2026 · LECTURE 05



东北大学悉尼智能科技学院 · 实习课程

MySQL进阶

—— DML 增删改查与多表查询

从 INSERT 到 SELECT 进阶，从单表到多表 —— 数据库真正的核心。

INSTRUCTOR · 讲师

靳老师

北京科迪诺亿科技有限公司
东北大学秦皇岛分校

01 / 35



翻页



编辑



全屏



INSERT: 往表里塞数据

列名和值一一对应，字符串用单引号，数字不用引号。

SQL · INSERT 标准语法

SQL

```
INSERT INTO 表名 (列名1, 列名2, ...)  
VALUES (值1, 值2, ...);
```

-- 示例: 插入一名学生

```
INSERT INTO student (name, age, gender, major, email)  
VALUES ('张三', 20, '男', '计算机', 'zhangsan@neuq.edu.cn');
```

- 字符串 / 日期 —— 用单引号 '张三'
- 数字 —— 不需要引号 20
- 有默认值或自增 —— 可以省略不写

简写：省略列名

插入所有列时可省列名，但要按建表顺序写值 —— 不推荐，改表结构会出错。

SQL · INSERT 简写

SQL

-- 简写形式 (不推荐)

INSERT INTO student **VALUES**

(**NULL**, '李四', 21, '女', '软件工程', 'lisi@neuq.edu.cn', **NULL**);

-- NULL 表示让 AUTO_INCREMENT / DEFAULT 自动填

为什么推荐标准写法 ▶ 表结构以后改了（加新列），简写法立刻报错。

一次插多行数据

用逗号分隔多条 VALUES —— 比循环单条插入快十倍。

SQL · 批量插入五名学生

SQL

```
INSERT INTO student (name, age, gender, major) VALUES
('张三', 20, '男', '计算机'),
('李四', 21, '女', '软件工程'),
('王五', 19, '男', '人工智能'),
('赵六', 22, '女', '数据科学'),
('孙七', 20, '男', '计算机');
```

注意 ▶ 多条数据用逗号分隔，最后一条后面加分号 ;。

SELECT: 90% 工作都是查询

* 表示所有列，但不推荐 —— 只查你需要的列更高效。

SQL · 查询

SQL

-- 查询所有列

```
SELECT * FROM student;
```

-- 查询指定列 (推荐)

```
SELECT name, age, major  
FROM student;
```

—— 不推荐 **SELECT *** —— 浪费性能

—— 表结构变了 —— * 结果也变，破坏代码

—— 类比 —— * 像点外卖不看价格全点，指定列像只点你要的

WHERE：精确过滤

只查满足条件的行 —— 六个常用运算符。

运算符	含义	示例
=	等于	WHERE age = 20
≠ / <>	不等于	WHERE major != '计算机'
>	大于	WHERE age > 20
≥ / ≤	大于/小于等于	WHERE score >= 60
LIKE	模糊匹配	WHERE name LIKE '张%'

SQL · 三种常用 WHERE

SQL

```
SELECT * FROM student WHERE age > 20;           -- 年龄大于 20
SELECT * FROM student WHERE major = '计算机';    -- 计算机专业
SELECT * FROM student WHERE name LIKE '张%';     -- 姓张的学生
```

AND = "而且" · OR = "或者"

组合条件时加括号更清晰，避免歧义。

SQL · AND / OR 组合

SQL

-- AND: 年龄大于 20 且 计算机专业

```
SELECT * FROM student WHERE age > 20 AND major = '计算机';
```

-- OR: 年龄小于 20 或 大于 22

```
SELECT * FROM student WHERE age < 20 OR age > 22;
```

-- 组合: 加括号

```
SELECT * FROM student  
WHERE (major = '计算机' OR major = '软件工程') AND age > 20;
```

IN：枚举 · BETWEEN：范围

比 OR 简洁，比 AND 范围直观。

SQL · IN / BETWEEN

SQL

-- IN：在指定列表中

SELECT * FROM student

WHERE major IN ('计算机', '软件工程', '人工智能');

-- 等价于: `major = '计算机' OR major = '软件工程' OR major = '人工智能'`

-- BETWEEN：在范围内（包含边界）

SELECT * FROM student WHERE age BETWEEN 19 AND 21;

-- 等价于: `age >= 19 AND age <= 21`

互动提问 ▶ BETWEEN 19 AND 21 包不包含 19 和 21？（答案：包含！）

NULL: 不能用 = 判断!

NULL 表示"未知" —— 必须用 IS NULL / IS NOT NULL。

SQL · IS NULL

SQL

-- 查询没填邮箱的学生

```
SELECT * FROM student WHERE email IS NULL;
```

-- 查询填了邮箱的学生

```
SELECT * FROM student WHERE email IS NOT NULL;
```

-- ❌ 错误! 永远查不到结果

```
SELECT * FROM student WHERE email = NULL;
```

类比 ▶ NULL 就像"未知" —— 你不能说一个未知的东西等于另一个未知的东西。

ORDER BY：让结果排队

ASC 升序（默认） / DESC 降序。多列排序：先按第一列，相同再按第二列。

SQL · 三种排序

SQL

```
-- 按年龄升序
SELECT * FROM student ORDER BY age ASC;

-- 按年龄降序
SELECT * FROM student ORDER BY age DESC;

-- 先按专业升序，再按年龄降序
SELECT * FROM student ORDER BY major ASC, age DESC;
```

类比 ▶ ORDER BY 像给成绩单排序 —— 可以按总分排，也可以先按班级排再按分数排。

LIMIT: App 翻页的底层

每个 App 的"下一页"功能，背后都是 LIMIT。

SQL · 分页三连

SQL

-- 第 1 页: 前 3 条

SELECT * FROM student LIMIT 3;

-- 第 2 页: 4-6 条 (偏移量 3, 取 3 条)

SELECT * FROM student LIMIT 3, 3;

-- 第 3 页: 7-9 条

SELECT * FROM student LIMIT 6, 3;

分页公式 ▶ LIMIT (页码 - 1) × 每页数量, 每页数量

DISTINCT: 去掉重复值

像取集合 —— 只保留不同的值。

SQL · 去重

SQL

```
-- 去重: 查询所有不同的专业
SELECT DISTINCT major
FROM student;

-- 不加 DISTINCT
SELECT major FROM student;
-- 计算机出现 3 次 (如果 3 个
-- 学生都是计算机专业)
```

MAJOR (去重后)				
计算机				
软件工程				
人工智能				
数据科学				

UPDATE: 改数据

SET 改值，WHERE 限定改哪些行 —— 可同时改多个字段。

SQL · UPDATE

SQL

-- 把张三的年龄改为 21

```
UPDATE student SET age = 21 WHERE name = '张三';
```

-- 同时改多个字段

```
UPDATE student SET age = 22, major = '软件工程' WHERE name = '李四';
```


忘 WHERE = 灾难现场

所有学生的专业都被改成"人工智能"!

SQL · 灾难演示

SQL

```
-- ✗ 忘了 WHERE!  
UPDATE student SET major = '人工智能';  
  
-- 结果: 所有学生的专业都变成了"人工智能"
```

真实案例 ▶ 某程序员执行 UPDATE user SET password = '123' (忘 WHERE)，所有用户密码全被重置，公司当晚全员加班修复。

安全措施 ▶ UPDATE / DELETE 永远加 WHERE！执行前先用 SELECT + 同样 WHERE 看看会影响哪些数据。

DELETE: 删除数据

永远要加 WHERE! 不加 WHERE 会删光整张表数据。

SQL · DELETE

SQL

-- 删除名字叫"孙七"的学生

```
DELETE FROM student WHERE name = '孙七';
```

-- 删除年龄小于 19 的学生

```
DELETE FROM student WHERE age < 19;
```

-- **✗** 危险! 会删除所有数据

```
DELETE FROM student;
```

TRUNCATE: 快速清空整表

两个差别，看场景选。

对比	DELETE FROM	TRUNCATE
速度	慢（逐行删）	快（直接清空）
WHERE	支持	不支持
自增重置	不重置	重置（id 从 1 开始）
事务回滚	可以	不可以
适用场景	删部分数据	清空整张表

类比 ▶ DELETE FROM 像一件一件把衣服从衣柜里拿出来；TRUNCATE 像把整个衣柜翻过来一次性倒空。

五个聚合函数：统计神器

COUNT 计数、SUM 求和、AVG 平均、MAX 最大、MIN 最小。

SQL · 单独使用

SQL

```
SELECT COUNT(*) FROM student;
SELECT SUM(score) FROM score;
SELECT AVG(score) FROM score;
SELECT MAX(score) FROM score;
SELECT MIN(score) FROM score;
```

SQL · 组合 + 别名

SQL

```
SELECT
    COUNT(*) AS 人数,
    AVG(score) AS 平均分,
    MAX(score) AS 最高分,
    MIN(score) AS 最低分
FROM score;
```

AS 别名 ▶ 给列取个易读的名字，结果更好看。

GROUP BY: 按班站队，再统计

按某列分组，每组单独做聚合统计。

SQL · 分组统计

SQL

-- 每个专业有多少人

```
SELECT major, COUNT(*) AS 人数
FROM student
GROUP BY major;
```

-- 多列分组：每个专业、每个性别多少人

```
SELECT major, gender, COUNT(*) AS 人数
FROM student
GROUP BY major, gender;
```

类比 ▶ GROUP BY 像把学生按专业分班站队，然后数每个班有多少人。

WHERE vs HAVING: 过滤时机不同

WHERE 在分组前过滤行，HAVING 在分组后过滤组。

WHERE

分组前的安检

- 过滤原始行
- 不能用聚合函数
- 类比：进门前的安检

HAVING

分组后的筛选

- 过滤分组结果
- 可以用聚合函数
- 类比：分好组后挑组

SQL · HAVING

-- 查询人数超过 1 人的专业

```
SELECT major, COUNT(*) AS 人数
FROM student
GROUP BY major
HAVING COUNT(*) > 1;
```

SQL

DML四件套

CRUD：Create / Read / Update / Delete。

语句	作用	示例
INSERT	插入	INSERT INTO student (name, age) VALUES ('张三', 20);
SELECT	查询	SELECT * FROM student WHERE age > 20;
UPDATE	更新	UPDATE student SET age = 21 WHERE name = '张三';
DELETE	删除	DELETE FROM student WHERE id = 5;

查询进阶 ▶ WHERE · ORDER BY · LIMIT · DISTINCT · 聚合 · GROUP BY + HAVING。

下半节：综合实操

单表查询是热身，多表查询才是真实业务场景。

NEXT

多表查询 + 子查询

INNER JOIN / LEFT JOIN / WHERE 子查询 / IN 子查询 + 图书管理系统综合练习。

► 课后作业：在 DBeaver 中练习今天的所有 SQL 语句

21

NEXT

查"张三的 Java 成绩"需要三张表

信息分散在三张表 —— 必须把表"连起来"查。

student 表 学号、姓名、年龄... 但没有成绩信息	course 表 课程编号、课程名... 但没有成绩信息	score 表 学号、课程编号、成绩 但学号课程号只是数字
---	--	---

解法 ▶ 用 JOIN 把三张表连起来，一条 SQL 搞定。

INNER JOIN：取两表交集

用 ON 指定连接条件 —— 只保留两表都能匹配的记录。

SQL · 三表 INNER JOIN

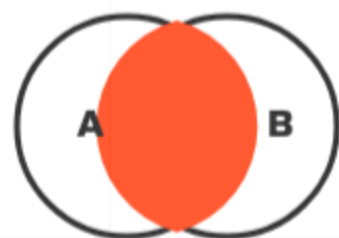
SQL

```
SELECT student.name, course.course_name, score.score
FROM score
INNER JOIN student ON score.student_id = student.id
INNER JOIN course ON score.course_id = course.id;
```

NAME	COURSE_NAME	SCORE
张三	Java	90.00
张三	数据库	85.00
李四	Java	78.00

两种 JOIN：交集 vs 全保留

INNER 取交集；LEFT 保留左表全部。



INNER JOIN

只保留两表都能匹配上的记录（红色交集）



LEFT JOIN

保留左表全部记录，右表匹配不上的显示 NULL

LEFT JOIN：一个都不漏

从左表出发，所有学生都显示 —— 没成绩的显示 NULL。

SQL · LEFT JOIN

SQL

```
SELECT student.name, course.course_name, score.score
FROM student
LEFT JOIN score ON student.id = score.student_id
LEFT JOIN course ON score.course_id = course.id;
```

NAME	COURSE_NAME	SCORE
张三	Java	90.00
李四	Java	78.00
王五	NULL	NULL

关键 ▶ 王五没有成绩，但也会出现在结果里 —— 这就是 LEFT JOIN 的威力。

三个经典业务查询

面试常考，工作常用。

SQL · 计算机专业学生成绩	SQL
<pre>SELECT s.name, c.course_name, sc.score FROM score sc INNER JOIN student s ON sc.student_id = s.id INNER JOIN course c ON sc.course_id = c.id WHERE s.major = '计算机';</pre>	
SQL · 每门课平均分	SQL
<pre>SELECT c.course_name, AVG(sc.score) AS avg_score FROM score sc INNER JOIN course c ON sc.course_id = c.id GROUP BY c.course_name;</pre>	
SQL · 最高分学生	SQL
<pre>SELECT s.name, c.course_name, sc.score FROM score sc INNER JOIN student s ON sc.student_id = s.id INNER JOIN course c ON sc.course_id = c.id ORDER BY sc.score DESC LIMIT 1;</pre>	

子查询：查询里的查询

把多步查询合并成一条 SQL。

生活类比 ▶ "找出 Java 成绩最高的人是谁？"

第一步：先查 Java 的最高分 → 95 分

第二步：再查谁的 Java 成绩是 95 → 张三

这两步合并成一条 SQL 就是子查询。

SQL · 子查询骨架

SQL

```
SELECT ... FROM ...  
WHERE 列 IN (  
  -- 这里是子查询  
  SELECT ... FROM ... WHERE ...  
);
```

嵌套三层：找到 Java 最高分学生

从最内层往外读，逐层解决问题。

SQL · 嵌套子查询

SQL

```
SELECT * FROM student
WHERE id IN (
    SELECT student_id FROM score
    WHERE course_id = (SELECT id FROM course WHERE course_name = 'Java')
    ORDER BY score DESC LIMIT 1
);
```

- 01 最内层 —— 找到 Java 课程的编号
- 02 中间层 —— 找到该课程最高分的学生编号
- 03 最外层 —— 查到学生的详细信息

IN/NOT IN: 在/不在结果里

子查询返回一组值，外查询判断在不在其中。

SQL · IN 子查询

SQL

-- 查询有不及格成绩的学生

```
SELECT DISTINCT name FROM student
WHERE id IN (
    SELECT student_id FROM score WHERE score < 60
);
```

-- 查询没选过课的学生

```
SELECT name FROM student
WHERE id NOT IN (SELECT DISTINCT student_id FROM score);
```


动手：图书管理系统

综合运用今天学的所有 SQL。

book 图书表

id, title, author, price, stock

reader 读者表

id, name, phone, level

borrow_record 借阅表

id, book_id, reader_id, borrow_date, return_date

步骤 ▶ 创建三张表 → 插入测试数据（5 本书、5 个读者、10 条借阅）→ 完成查询任务。

四部分挑战任务

30 分钟通关。

部分	时长	任务
第一部分	10 min	建三表 + 插入测试数据
第二部分	5 min	单表查询（贵书、库存 0、前 3 本）
第三部分	10 min	多表查询（每人借几本、张三借了啥、未被借的书）
第四部分	5 min	聚合分组（每类平均价、每月借阅量）

三个经典查询参考答案

SQL · 参考答案

SQL

```
-- 1. 每个读者借了几本书
SELECT r.name, COUNT(*) AS borrow_count
FROM borrow_record br
INNER JOIN reader r ON br.reader_id = r.id
GROUP BY r.name;

-- 2. 张三借了哪些书
SELECT b.title, br.borrow_date
FROM borrow_record br
INNER JOIN book b ON br.book_id = b.id
INNER JOIN reader r ON br.reader_id = r.id
WHERE r.name = '张三';

-- 3. 哪些书没被借过 (LEFT JOIN 找 NULL)
SELECT b.title FROM book b
LEFT JOIN borrow_record br ON b.id = br.book_id
WHERE br.id IS NULL;
```


五个常见报错

看懂错误信息，问题就解决了一半。

错误	原因	解决
Table doesn't exist	表名写错 / 没选数据库	检查表名，USE 数据库
Unknown column	列名写错	检查列名拼写
Syntax error near ...	SQL 语法错	看 near 后的位置
Duplicate entry	主键 / 唯一键冲突	检查数据是否重复
Incorrect string value	字符集问题	确认 utf8mb4 编码

写出优雅的 SQL

不是强制，但专业程序员都这么做。

✓ 推荐写法

SQL

```
SELECT name, age, major
FROM student
WHERE age > 20
ORDER BY age DESC;
```

✗ 不推荐

SQL

```
select name,age,major from
student where age>20 order
by age desc;
```

- 关键字大写 —— SELECT、FROM、WHERE
- 表名列名小写 —— 用下划线分隔 student_name
- 每子句一行 —— 缩进对齐
- 注释用 -- —— 或 /* */

MySQL 两节课全部通关

下节课进入 SpringBoot —— 让数据库和 Web 服务连接起来。

01

环境安装

MySQL + DBeaver + 第一个数据库。

02

DDL

数据类型 + 建表 + 改表 + 删表。

03

DML

增删改查 + 排序 + 分页 + 聚合 + 分组。

04

综合实操

多表 JOIN + 子查询 + 图书管理系统。

NEXT

下节预告 · SpringBoot 起步

让数据库和 Web 服务连接起来 —— 真正进入后端开发。

► MyBatis-Plus = 自动挡汽车（不用手写 SQL）

35

FINAL RECAP